

Prompts Documentation

1. So, I will go with Track 1
As I have only the free versions of all the ais
So, PLEASE HELP ME WITH THE AI that I can USE to maximize the output for the track 1
Like choosing the best ai tool for ui,
best tool for prompt to give the other ai to make them do the best work

AND also tell me that HOW CAN I USE ALL THE AIS AND LINK EACH OTHER TO GENERATE THE BEST OUTPUT
I know you are the best mentor
2. Now tell me what's our next step?
3. what was this step about?
4. move forward with the next step
5. As I am thinking to convert everything to antigravity
So please give me the settings and models to choose so that I can make full use of antigravity
I want to automate everything with antigravity
6. I have done nothing work on this project just selected the track 1 only after that i am just reading your chats so please tell me what should i have to do and what all things I have to provide the antigravity to work
7. You are the Lead AI Product Architect, Research Engineer, and Technical Project Manager for my individual KOHLER-MITWPU AI Research Lab case study.
IMPORTANT:
This is the FIRST project-planning conversation.

Do NOT write application code yet.
Do NOT create the frontend yet.
Do NOT create the backend yet.
Do NOT install dependencies yet.
Do NOT create API keys yet.
Do NOT scrape websites yet.
Do NOT make irreversible project changes.

Your task in this conversation is ONLY to understand the challenge deeply, perform project discovery, challenge assumptions, and produce a rigorous implementation blueprint.

=====

1. COMPETITION

=====

Organization:

KOHLER-MITWPU AI Research Lab Program

This is an individual AI case-study challenge.

Submission deadline:

September 20, 2026 at 11:59 PM IST.

Submission must be through a single GitHub repository.

Required submission contents:

1. Working Model

- Source code
- Instructions/scripts to run the functional prototype

2. Prompts Documentation PDF

- AI prompts
- System instructions
- AI workflows used

3. Video Demonstration

- 1–3 minute walkthrough

4. Presentation Deck

- PDF
- Maximum 4 slides
- Core approach
- Architecture
- Tech stack
- Innovation pitch

Evaluation:

Approach & Innovation — 45%

Technical Execution — 25%

User Experience & Feasibility — 20%

Business & Sustainability Impact — 10%

=====

2. SELECTED TRACK

=====

TRACK 1 — KOHLER AI Bathroom Designer & Planner

Official objective:

Build an interactive AI design assistant that takes customer constraints and automates personalized product bundle recommendations.

Key inputs include:

- Bathroom dimensions
- Bathroom layout or image
- Budget limits
- Aesthetic themes such as Minimalist Modern, Classic Luxury, Japanese Zen
- KOHLER device/product catalog specifications

Expected outcome:

An intelligent recommendation engine that outputs optimized product combinations such as:

- Faucets
- Smart toilets
- Thermostatic showers
- Vanities

The combinations should fit physical-space and budget parameters.

If possible:

- 2D bathroom representation
- 3D representation

=====

3. MY CURRENT STATUS

=====

I have done essentially NO implementation work yet.

I have only:

- selected Track 1
- created/opened this Antigravity project
- configured the Antigravity security settings

Therefore, assume this is a brand-new project.

Do not assume that any architecture, code, database, product catalog, API integration, or UI already exists.

=====

4. MY RESOURCE CONSTRAINTS

=====

I am an individual participant.

I want to maximize the use of free AI models and free service tiers.

I want Antigravity to automate as much of the development lifecycle as reasonably possible.

The project must be realistic to complete before the submission deadline.

Avoid unnecessary:

- paid APIs
- paid infrastructure
- excessive dependencies
- unnecessary microservices
- unnecessary multi-agent complexity
- custom model training unless clearly justified

Prefer:

- modular architecture

- replaceable services
- free/open technologies where practical
- deterministic logic for critical calculations
- strong validation
- reproducible development

=====

5. OUR INITIAL PRODUCT DIRECTION

=====

Our concept is NOT a generic AI chatbot.
The proposed product is:

KOHLER AI Bathroom Designer & Planner

A multimodal AI-powered bathroom design optimization system that accepts:

- bathroom dimensions
- optional bathroom image/layout
- budget
- aesthetic preferences
- functional requirements
- sustainability priorities

and produces:

- physically feasible product bundles
- budget-aware recommendations
- aesthetic/style matching
- functional compatibility
- sustainability/water-efficiency analysis
- explainable recommendations
- a 2D bathroom layout

The conceptual pipeline is:

Customer Input

- Multimodal AI Understanding
- Structured Requirements
- Grounded KOHLER Product Retrieval
- Hard Constraint Filtering
- Multi-Objective Optimization
- Sustainability Analysis
- Explainable Recommendation
- 2D Bathroom Layout

=====

6. CORE ARCHITECTURAL PRINCIPLE

=====

Do NOT design this as:

User → LLM → Product Recommendation

Instead, the architecture should separate:

1. AI interpretation
2. Product-data retrieval

3. Hard constraints
4. Optimization
5. Sustainability calculations
6. Explanation
7. Visualization

The LLM must NOT independently invent critical product facts.

Never fabricate:

- product prices
- dimensions
- product compatibility
- water-consumption values
- installation requirements
- electrical requirements
- product availability

Critical product facts must come from a controlled product catalog or verified source.

Deterministic application logic must enforce hard constraints.

=====

7. PROPOSED AI TOOLING DIRECTION

=====

Antigravity should be the primary development environment.

Likely AI/tool roles are:

Claude:

- architecture
- coding
- refactoring
- debugging
- repository review
- testing
- security review

Gemini:

- multimodal image understanding
- structured requirement extraction
- Gemini API runtime intelligence
- natural-language explanation
- Google ecosystem integration where appropriate

Fast/cheaper models:

- routine edits
- boilerplate
- simple tasks
- formatting
- repetitive changes

However, do NOT blindly accept these assignments.

Evaluate them and recommend the best model-routing strategy based on:

- quality
- free-tier limits
- task complexity
- reliability

=====

8. INITIAL EXPECTED SYSTEM

=====

We are considering a hybrid architecture approximately like:

USER

|

+-- Text requirements

+-- Dimensions

+-- Bathroom Image

|

v

Multimodal AI

|

v

Structured User Intent

|

v

Product Retrieval

|

v

Controlled KOHLER Product Catalog

|

v

Hard Constraint Engine

|

+-- Space

+-- Budget

+-- Compatibility

+-- Functional requirements

|

v

Multi-Objective Optimization

|

+-- Budget Fit

+-- Spatial Fit

+-- Style Match

+-- Functionality

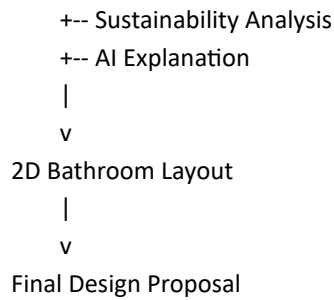
+-- Sustainability

|

v

Top Product Bundles

|



Treat this as a starting hypothesis, NOT a frozen architecture.

Challenge it.

Improve it.

Explain what should change and why.

9. INITIAL MVP IDEA

A realistic MVP may support:

INPUT:

- Bathroom dimensions
- Budget
- Style/theme
- Required products/features
- Optional bathroom image

PROCESSING:

- Requirement extraction
- Image understanding
- Product retrieval
- Hard constraint validation
- Product bundle optimization
- Sustainability scoring

OUTPUT:

- Recommended bundle
- Product list
- Total cost
- Budget utilization
- Space compatibility
- Style score
- Sustainability score
- Estimated water impact/savings
- Explanation
- 2D bathroom layout

INTERACTION:

User can modify a priority such as:

"Make it more sustainable."

The system should re-optimize instead of merely generating another paragraph.
Again, this MVP is a hypothesis. Evaluate it and improve it.

=====

10. IMPORTANT TECHNICAL REQUIREMENT

=====

Distinguish between:

HARD CONSTRAINTS
and
SOFT PREFERENCES.

Hard constraints may include:

- Bundle cannot exceed budget
- Required product category must be satisfied
- Product must fit available space
- Installation requirements must be satisfied
- Required electrical conditions must be satisfied
- Required functionality must be satisfied

Soft preferences may include:

- Japanese Zen
- Minimalist
- Luxury
- Water efficiency
- Smart features
- Finish preferences

A solution that violates a hard constraint must not be recommended simply because it has a high aesthetic score.

=====

11. DATA STRATEGY

=====

We expect to create a curated KOHLER product catalog for the prototype rather than ingesting the entire catalog.

Approximate initial target:

40–60 well-verified products.

Possible categories:

- Toilets
- Faucets
- Showers
- Vanities/basins

Potential attributes:

- product ID

- model number
 - product name
 - category/subcategory
 - market
 - price
 - dimensions
 - installation requirements
 - rough-in
 - electrical requirements
 - water consumption / flow rate
 - smart features
 - style tags
 - finish
 - collection/family
 - compatibility
 - space requirements
 - sustainability information
 - source URL
 - verification date
 - confidence metadata
- Do not invent missing product fields.

=====

12. SUSTAINABILITY

=====

Sustainability is not a decorative feature.
It should be part of the recommendation methodology.
Explore how the system can calculate:

- water-efficiency score
- estimated water consumption
- estimated water savings relative to a defined baseline
- sustainability score

All calculations must expose their assumptions.
Do not claim precise real-world savings without specifying the usage assumptions and baseline.

=====

13. VISUALIZATION

=====

For the MVP, prioritize a polished 2D bathroom planner over a weak 3D implementation.
Possible 2D elements:

- room boundaries
- door
- windows
- toilet
- shower
- vanity
- selected products
- labels

- approximate dimensions

3D can be considered a stretch goal only if it does not endanger the core MVP.

=====

14. ANTIGRAVITY AUTOMATION

=====

I want Antigravity to automate as much as reasonably possible.

Evaluate how Antigravity can help with:

- project scaffolding
- coding
- database setup
- data validation
- tests
- browser testing
- debugging
- documentation
- git operations
- build verification
- deployment preparation
- regression testing
- submission validation

Also identify which decisions MUST remain human-approved.

Do not assume unrestricted autonomy is always desirable.

=====

15. REQUIRED OUTPUT

=====

Produce a rigorous project-discovery report with the following sections:

1. Executive Summary
2. Problem Understanding
3. Recommended Product Vision
4. User Personas / Primary User Journey
5. Core User Stories
6. Functional Requirements
7. Non-Functional Requirements
8. Innovation Analysis
9. Competitive Differentiation
10. Recommended Technical Architecture
11. AI Architecture
12. Model-Routing Strategy
 - which model for which task
 - why
 - free-tier considerations
13. Product Database Architecture
14. Product Data Schema

15. Product Retrieval Strategy
16. Constraint Engine Design
17. Multi-Objective Optimization Strategy
18. Sustainability Methodology
19. Multimodal Bathroom Image Analysis Strategy
20. 2D Layout Architecture
21. Frontend Architecture
22. Backend Architecture
23. API Architecture
24. Security Architecture
25. Validation / Hallucination Prevention
26. Testing Strategy
27. Browser Automation Strategy
28. Antigravity Agent / Subagent Strategy
29. Antigravity Rules and Skills Strategy
30. External Accounts / APIs / Services Required
31. Free-Tier Feasibility Analysis
32. MVP Scope
33. Stretch Features
34. Major Risks
35. Risk Mitigation
36. Expected Development Effort
37. Day-by-Day Plan Until September 20, 2026
38. Recommended GitHub Repository Structure
39. Required Documentation
40. Demo Strategy
41. 4-Slide Presentation Strategy
42. Prompt Documentation PDF Strategy
43. Final Submission Checklist
44. Open Questions
45. Decisions Requiring Human Approval

=====

16. CRITICAL THINKING REQUIREMENT

=====

Do not simply agree with my proposed architecture.

Act as a skeptical senior architect.

Identify:

- unnecessary complexity
- unrealistic assumptions
- hidden dependencies
- free-tier risks
- AI hallucination risks
- computer-vision limitations
- data-quality problems
- optimization problems
- UX issues

- deadline risks

For every major recommendation, explain WHY.

When there are multiple approaches, compare them and recommend one.

=====

17. CURRENT PROJECT STATE

=====

The repository should be treated as effectively empty/new.

Do not assume existing source files.

Do not generate the production application yet.

=====

18. FINAL INSTRUCTION

=====

This is a PLANNING ONLY task.

Do not write application code.

Do not install dependencies.

Do not create API keys.

Do not create external services.

Do not scrape KOHLER data.

Do not make irreversible project changes.

At the end, give me:

A. Your recommended final architecture.

B. Your recommended MVP.

C. Your recommended free-tier technology stack.

D. Your recommended model-routing strategy.

E. The exact information/accounts/assets that I must provide before implementation begins.

F. The exact first implementation task we should execute after this planning phase.

Stop after producing the plan and wait for my approval.

8. I have reviewed your implementation blueprint.

The overall architecture is approved in principle, but DO NOT begin implementation yet.

Before implementation, revise the blueprint with these mandatory changes:

1. Runtime Gemini:

Replace Gemini 2.0 Flash with the currently supported stable Gemini API model.

Prefer gemini-2.5-flash for the MVP runtime because it supports multimodal input and structured outputs and keeps the runtime architecture simple.

Use the same runtime model for image/requirement analysis and explanation unless there is a strong technical reason not to.

2. Date correction:

Correct the day names in the schedule:

September 15, 2026 = Tuesday

September 16 = Wednesday

September 17 = Thursday

September 18 = Friday
September 19 = Saturday
September 20 = Sunday and submission deadline.

3. Spatial validation:

Do NOT claim that summing product footprint areas proves physical feasibility.

Use a template/zone-based layout feasibility model.

Enforce room boundaries, product non-overlap, assigned fixture zones, and defined circulation/clearance rules.

Explicitly describe this as a prototype spatial-feasibility model, not a complete CAD/architectural solver.

4. Rough-in:

Do not enforce rough-in as a hard constraint unless the user supplies rough-in information.
Otherwise show installation requirements as warnings/information.

5. Sustainability:

Keep EPA WaterSense as an external benchmark only.
Do not describe EPA values as Indian federal standards.
Present water usage primarily in liters.
Explicitly label all results as estimates based on stated assumptions.
For dual-flush products, calculate expected consumption using an explicit full-flush/reduced-flush usage assumption.

6. Budget scoring:

Remove the assumption that the optimal bundle should use approximately 80% of the budget.
Budget should primarily be a hard upper limit.

Soft scoring should reflect affordability/value and user-selected priorities such as lower-budget, balanced, or luxury.

7. Product catalog:

Change the initial target to 32–40 verified products.
Start with approximately:
8 toilets
8 faucets
8 showers
8 vanities
Add more only after the MVP is stable.

8. Fallback:

Remove Groq as an initial dependency.

If Gemini fails, the deterministic recommendation engine must still work and the system should use a template-based explanation fallback.

9. Keep the following architecture:

- AI interpretation
- verified product catalog
- deterministic hard constraints
- weighted optimization
- deterministic sustainability calculation
- AI explanation
- 2D visualization.

10. Maintain the rule:

The LLM must NEVER invent product price, dimensions, water data, compatibility or installation specifications.

11. Recalculate the day-by-day plan after these changes.

We need a working prototype first, documentation second, stretch features last.

Return the revised implementation blueprint and a final "APPROVED ARCHITECTURE" section.

Do not write application code yet.

Do not install dependencies yet.

Do not create external services yet.

Wait for my approval after the revised blueprint.

9. Before implementation, make these final consistency corrections to the implementation blueprint.

1. Replace EVERY remaining reference to Gemini 2.0 Flash with Gemini 2.5 Flash.

Gemini 2.5 Flash is the single runtime model for:

- image analysis
- requirement extraction
- explanation
- re-optimization explanation

2. Replace EVERY remaining reference to 40–60 products or 48 products with:

32–40 verified KOHLER India products.

Initial target:

- 8 toilets
- 8 faucets
- 8 showers
- 8 vanities

3. Mark these decisions FINAL:

- Architecture: Approved
- Tech stack: Approved
- Market: India / INR
- Product catalog: 32–40
- MVP: Approved
- Image analysis: Stretch feature

- Budget: ₹50,000—₹10,00,000

4. Remove obsolete contradictory references from the appendices, tables, summaries, diagrams and model-routing sections.

5. Confirm the final pipeline is:

AI interpretation

→ verified product catalog

→ deterministic hard constraints

→ weighted optimization

→ deterministic sustainability

→ AI explanation with template fallback

→ 2D visualization.

6. Do NOT change the approved architecture.

7. Do NOT install anything.

8. Do NOT create application code.

9. Do NOT create accounts or API keys.

After correcting the document, clearly state:

"ARCHITECTURE READY FOR IMPLEMENTATION"

Then stop and wait.

10. all my model usage is over till 22 September because of and now tell me what should I do
[with the screenshot of all the existed models in antigravity]

11. [started doing manual work and provided all the output of the terminal to chatgpt one by one]

12. ONE FINAL CHECK

please go through all the requirements for this track

[pasted all requirements stated in the mail]

I think we are lacking in the ui part as there is no animation and color in the website and also there no way we can actually identify a particular button

13. Yes, please help me to achieve all this please

14. I want claude to this for me instead of antigravity as all the model limits are exhausted in the antigravity

We are doing the FINAL REQUIREMENT-CLOSURE + UI POLISH pass for the KOHLER AI Bathroom Designer & Planner.

IMPORTANT:

- Do NOT redesign the architecture.
- Do NOT remove any existing working functionality.
- Do NOT replace the deterministic recommendation engine with AI.
- Preserve the current working Gemini integration, optimizer, sustainability engine, layout engine, APIs, tests, and existing catalog.
- Make incremental changes only.
- After all changes, run lint, tests, and production build.
- Fix any errors before finishing.

PROJECT GOAL:

The Track 1 requirement is:

"Build an interactive AI design assistant that takes a customer's constraints and automates personalized product bundle recommendations."

Required inputs:

- Bathroom dimensions
- Bathroom layout/image
- Budget
- Aesthetic theme
- Product catalog specifications

Expected output:

- Optimized product combinations covering:
 1. Faucets
 2. Smart toilets
 3. Thermostatic showers
 4. Vanities
- Products must fit physical-space and budget constraints
- 2D/3D representation if possible

CURRENT APP:

- Next.js 15 App Router
- TypeScript
- Tailwind CSS
- shadcn/ui
- react-konva
- Zustand
- Zod
- Gemini
- Deterministic TypeScript optimizer
- Current build is already passing
- Current catalog has 8 products
- Current categories include toilet, faucet, shower, vanity
- Existing recommendation engine must remain deterministic

TASK 1 — CLOSE THE PRODUCT-CATEGORY GAP

Add at least:

- A) One verified KOHLER India smart/intelligent toilet
- B) One verified KOHLER India thermostatic shower/system

Use real KOHLER India product information and official KOHLER India product pages.

Preferred smart toilet candidate:

- KOHLER Innate K-29777IN-0
- Verify current official KOHLER India product information before adding it.

Preferred thermostatic shower candidate:

- KOHLER Anthem thermostatic system
- Prefer K-26341IN-NA or another appropriate current KOHLER India thermostatic shower/system.
- Verify official KOHLER India information before adding it.

Do NOT invent:

- price
- dimensions
- water consumption
- electrical requirements
- installation requirements
- compatibility

If an important field cannot be confidently verified, use null/appropriate confidence metadata rather than fabricating it.

Extend the existing product types only as much as necessary.

Use the existing:

- dataConfidence
- verificationDate
- productUrl
- dimensions
- waterConsumption
- installation information

Do not break the Zod schemas.

The smart toilet should belong to the existing toilet category.

The thermostatic shower should belong to the existing shower category, but preserve enough metadata to distinguish it from a simple showerhead/handshower.

If necessary, add a field such as:

- showerType: "showerhead" | "handshower" | "thermostatic-system"
- or an equivalent minimal typed field.

Do not create a fake floor footprint for a thermostatic valve.

Represent the thermostatic system as part of the shower zone / shower fixture system.

TASK 2 — PRESERVE AND VERIFY OPTIMIZATION

Update candidate filtering/scoring only where necessary.

The optimizer must still:

- enforce budget as a hard ceiling
- enforce category coverage
- enforce product/zone dimensional fit
- preserve existing deterministic scoring
- preserve sustainability calculations
- return top ranked bundles

Make sure adding the new products does not cause the existing working recommendations to fail.
If the catalog now contains multiple toilets/showers, the optimizer must actually be able to choose among them.

Add/adjust tests for:

- smart toilet included in valid candidate pool
- thermostatic shower included in valid candidate pool
- budget constraint still works
- optimizer can return a bundle containing the new product classes

TASK 3 — IMPROVE PRIMARY UI ACTIONS

Open the current main page implementation and improve the visual hierarchy without rewriting the whole page.

The primary action should be unmistakable:

"Generate My Bathroom"

Requirements:

- larger than secondary buttons
- clear high-contrast treatment
- recognizable icon
- strong hover state
- pressed/active state
- focus-visible state
- disabled state
- loading state

When generating, show an informative state such as:

"Optimizing your bathroom..."

or

"Generating your KOHLER design..."

Do NOT use excessive animations.

TASK 4 — IMPROVE BUTTON IDENTIFICATION

Every important action should have a clear icon and visual hierarchy.

Use Lucide/shadcn-compatible icons where available.

Examples:

- AI interpretation → Sparkles
- Upload image → Image
- Analyze image → Scan/Search
- Generate → Wand/Sparkles/ArrowRight
- Product details → ExternalLink
- Sustainability → Droplets
- Layout → LayoutGrid/Ruler

Make sure text labels remain present.

Do not rely on icons alone.

Add:

- hover transition
- focus ring
- active/pressed state
- disabled state where applicable

TASK 5 — ADD SUBTLE PRODUCTION-QUALITY ANIMATION

Use minimal tasteful motion:

- button hover transitions
- loading spinner
- results fade/slide-in
- recommendation cards subtle staggered entrance
- sustainability number/value subtle reveal

Avoid:

- excessive motion
- large bouncing effects
- distracting animations
- animation on every element

Respect prefers-reduced-motion if practical.

TASK 6 — IMPROVE COLOR HIERARCHY

Do not redesign the entire theme.

Use a restrained premium bathroom/product aesthetic:

- warm/neutral page background
- clean white cards
- strong primary brand/accent color
- green/teal semantic treatment for sustainability
- amber for warnings
- red for errors

Keep text highly readable.

Make the sustainability result visually distinct, e.g.:

"WATER SAVED"

"81,030 L / year"

"Estimated under your stated usage assumptions"

Do not claim exact real-world savings.

Keep the existing assumptions/disclaimer.

TASK 7 — IMPROVE WORKFLOW CLARITY

Near the main input area, add a simple visual workflow indicator:

01 Requirements

02 Optimization

03 Your Design

This should be lightweight and not consume excessive space.

Do not add a complex multi-page wizard.

TASK 8 — IMPROVE RESULT HIERARCHY

Make it visually obvious:

- recommended bundle
- total price
- budget utilization
- recommendation score
- water impact
- 2D layout

The primary recommended bundle should visually stand out from alternatives without assigning a new ranking system that changes the optimizer.

Keep product-level links.

TASK 9 — ACCESSIBILITY

Check:

- buttons have readable labels
- keyboard focus is visible
- form controls have associated labels
- image upload has accessible labeling
- icons do not replace meaningful text
- loading states are understandable
- contrast is reasonable
- interactive elements are keyboard accessible

TASK 10 — RESPONSIVENESS

Check desktop and narrower viewport behavior.

Do not let:

- buttons overflow

- cards break
- layout canvas destroy the page width
- text become unreadable

TASK 11 — DO NOT BREAK CURRENT FEATURES

Verify that these still work:

1. Natural-language AI interpretation
2. Manual requirements
3. Image upload
4. Image analysis
5. Theme prefill
6. Product recommendation
7. Sustainability calculation
8. 2D bathroom layout
9. KOHLER product links
10. No-feasible-results handling

TASK 12 — FINAL VALIDATION

Run exactly:

```
npm run lint
npm test
npm run build
```

If any fail:

- diagnose
- fix
- rerun until all pass

Do NOT finish while there are build/type/lint/test errors.

Also verify there are no accidental secrets committed or exposed in client-side code.

IMPORTANT FINAL REPORT:

After making the changes, report:

1. Files changed
2. New KOHLER smart toilet added
3. New thermostatic shower added
4. UI improvements made
5. Tests passed
6. Build status
7. Any limitation that remains

DO NOT rewrite unrelated files.

DO NOT change the core AI-vs-deterministic architecture.

DO NOT invent product specifications.

DO NOT add unnecessary dependencies unless absolutely required.

15. I need you to modify the existing GitHub repository:

kohler-ai-bathroom-designer

Repository:

<https://github.com/DivyanshAgarwal7/kohler-ai-bathroom-designer>

You are working on the existing codebase, not creating a new project.

FIRST: inspect the repository thoroughly before making changes.

Review at minimum:

- package.json
- app/page.tsx
- app/globals.css
- app/api/*
- components/*
- components/ui/*
- data/products.ts
- data/schemas.ts
- data/categories.ts
- data/styles.ts
- data/sustainability-baselines.ts
- types/product.ts
- types/recommendation.ts
- lib/ai/*
- lib/engine/*
- lib/layout/*
- __tests__/*

Understand the current implementation and preserve it.

```
=====
PROJECT
=====
```

This is the KOHLER AI Bathroom Designer & Planner for Track 1.

Current architecture:

User input

- Gemini interprets text/image
- structured requirements
- verified KOHLER product catalog
- deterministic hard constraints
- deterministic weighted optimization
- deterministic sustainability calculation
- explanation
- 2D bathroom visualization

This architecture is intentional.

The key rule is:

AI understands the customer.

Deterministic logic makes the recommendation.

Do NOT replace deterministic product selection, constraint checking, sustainability calculation, scoring, or layout logic with an LLM.

=====
CURRENT STATE
=====

The application is already working.

Existing functionality includes:

- natural-language requirement interpretation
- bathroom image analysis
- manual dimension/budget/style inputs
- KOHLER product catalog
- deterministic recommendation engine
- hard budget constraints
- spatial/zone validation
- sustainability calculations
- 2D bathroom layout
- product-level KOHLER links
- no-feasible-result handling
- API routes
- automated tests

Do not remove or regress any of these.

The current production build is already passing.

=====
TASK 1 — CLOSE TRACK 1 PRODUCT-CATEGORY GAP
=====

The Track 1 expected product output explicitly mentions:

1. Faucets
2. Smart toilets
3. Thermostatic showers
4. Vanities

The current catalog has toilet, faucet, shower, and vanity products, but I need explicit coverage for:

A. One verified KOHLER India smart/intelligent toilet.

Preferred candidate:

KOHLER Innate K-29777IN-0

B. One verified KOHLER India thermostatic shower/system.

Preferred candidate:

KOHLER Anthem thermostatic system, preferably K-26341IN-NA or another appropriate current KOHLER India thermostatic product.

Use official KOHLER India sources where available.

CRITICAL:

Never invent:

- price
- dimensions
- rough-in
- water consumption
- electrical requirements
- compatibility
- installation requirements

If a field cannot be confidently verified:

- use null where the existing schema permits
- use the existing dataConfidence mechanism
- add an appropriate warning/metadata
- do not fabricate data

Preserve:

- dataConfidence
- verificationDate
- productUrl
- dimensions
- installation
- waterConsumption
- smartFeatures

The smart toilet should remain:

category = toilet

For showers, minimally distinguish:

- showerhead
- handshower
- thermostatic-system

Only extend the type/schema if genuinely necessary.

Do NOT create a fake floor footprint for a thermostatic valve.

Represent the thermostatic system as being associated with the shower zone/system.

=====

TASK 2 — VERIFY OPTIMIZATION

=====

The optimizer must continue to:

- enforce total budget $\leq$ user budget
- enforce required category coverage
- enforce product/zone dimensional fit
- preserve deterministic scoring
- preserve sustainability calculations
- return top-ranked bundles

The new smart toilet and thermostatic shower must participate naturally in optimization.
Do NOT force them into every recommendation.

Add/modify tests so that:

- smart toilet is present in valid catalog
- thermostatic shower is present in valid catalog
- optimizer can consider them
- budget constraint still works
- category coverage still works
- existing space-fit logic still works
- valid bundles can include the new product classes

=====

TASK 3 — PRODUCTION-QUALITY UI POLISH

=====

Do NOT redesign the whole application.

Improve the current UI incrementally.

PRIMARY CTA:

Make this visually unmistakable:

"Generate My Bathroom"

It must have:

- stronger visual hierarchy than secondary buttons
- recognizable icon
- hover state
- active/pressed state
- focus-visible state
- disabled state
- loading state

Use wording such as:

"Optimizing your bathroom..."
during recommendation generation.

=====

TASK 4 — BUTTON DISCOVERABILITY

=====

Use suitable existing icon components where possible.

Examples:

AI interpretation → Sparkles

Upload image → Image

Analyze image → Scan

Generate → Wand/Sparkles

Product details → ExternalLink

Water/sustainability → Droplets

Layout → LayoutGrid/Ruler

Keep text labels.

Do not rely on icons alone.

Use clear:

- primary
- secondary
- tertiary

button hierarchy.

=====

TASK 5 — SUBTLE ANIMATION

=====

Add restrained professional motion:

- button hover transitions
- loading spinner
- subtle result entrance
- subtle recommendation-card entrance
- subtle sustainability-value reveal

Do NOT add:

- excessive bouncing
- flashy effects
- unnecessary animation everywhere
- heavy animation libraries

Respect prefers-reduced-motion where practical.

=====

TASK 6 — COLOR HIERARCHY

=====

Keep the existing general design language.

Improve hierarchy with a restrained premium bathroom aesthetic:

- warm/neutral background
- clean cards
- strong primary accent
- green/teal sustainability treatment
- amber warnings
- red errors

Keep text readable.

Make the water-savings information visually distinct:

WATER SAVED

81,030 L / year

Estimated under your stated usage assumptions

Do not change the underlying sustainability calculations.

=====

TASK 7 — WORKFLOW INDICATOR

=====

Near the main input section add a compact indicator:

01 Requirements

02 Optimization

03 Your Design

Do not turn the app into a multi-page wizard.

=====

TASK 8 — RESULT HIERARCHY

=====

Make the following immediately visible:

- recommended bundle
- total price
- budget utilization
- recommendation score
- water impact
- 2D layout

Make the primary recommendation visually distinct from alternatives.

Do not create a new ranking/scoring methodology.

Keep product-level links.

=====

TASK 9 — ACCESSIBILITY

=====

Check:

- labels for form fields
- accessible image upload
- readable button text
- visible keyboard focus
- keyboard accessibility
- understandable loading states
- reasonable color contrast
- icons not being the only semantic indicator

=====

TASK 10 — RESPONSIVENESS

=====

Verify the current UI at:

- desktop
- tablet
- narrow mobile

Fix only genuine issues.

Ensure:

- buttons don't overflow
- cards don't break
- text stays readable
- layout stays contained
- form remains usable

=====

TASK 11 — PRESERVE EVERYTHING

=====

Verify all existing functionality remains:

1. Natural-language AI interpretation
2. Manual requirements
3. Image upload
4. Image analysis
5. Theme prefill
6. Product recommendation
7. Sustainability calculation
8. 2D bathroom layout
9. KOHLER product links
10. No-feasible-results handling

Do not remove or bypass any feature.

=====

TASK 12 — CODE QUALITY

=====

Prefer existing project components and utilities.
Do not add dependencies unless necessary.
Do not rewrite unrelated files
Keep TypeScript strongly typed.

Do not introduce `any` unnecessarily.
Keep the AI/deterministic boundary intact.

=====

TASK 13 — VALIDATION

=====

After modifications, verify:

```
npm run lint
npm test
npm run build
```

Fix all errors.

The final project must have:

- no lint errors
- no TypeScript errors
- no failing tests
- no build errors

Also verify:

- `.env.local` remains ignored
- API keys/secrets are not exposed
- no accidental generated files are added

=====

IMPORTANT

=====

Do not blindly change code.

Inspect first.
Modify only what is necessary.
Preserve existing working behavior.
Do not invent KOHLER product data.
Do not change the core architecture.
Do not add unrelated features.

When finished, provide a concise report containing:

1. Files modified
2. Smart toilet added
3. Thermostatic shower added
4. Type/schema changes
5. Optimizer/test changes
6. UI improvements
7. Accessibility improvements
8. Responsive improvements
9. Lint result
10. Test result
11. Build result
12. Remaining limitations

Do NOT automatically make unrelated repository changes.

- 16.** this is the output that Claude has generated
please check it tell me should I accept this output completely or not
[pasted the output of the Claude in the Chatgpt]

- 17.** Before I accept the patch, make these three small corrections.

1. THERMOSTATIC SHOWER MODELING

The Anthem K-26341IN-NA is specifically a recessed mechanical thermostatic valve, and KOHLERstates that the three-outlet control-panel trim is sold separately.

Do not let the UI imply that K-26341IN-NA by itself is a complete shower fixture.

Keep K-26341IN-NA in the catalog, but make its presentation explicit:

"Anthem Three-Port Thermostatic Valve — component of thermostatic shower system"

The recommendation/product card should clearly disclose:

"Valve body only — compatible control-panel trim sold separately."

Do not fabricate a floor footprint.

Also make the test wording reflect that this is a thermostatic-system component rather than a complete showerhead.

Do not redesign the optimizer unless necessary.

2. THERMOSTATIC PRODUCT FEATURE FIELD

The new s3 record currently uses:

smartFeatures: ['antiscald-protection']

That field is semantically intended for smart/electronic features.
Do not classify a mechanical thermostatic valve as a smart product.

Change this to:
smartFeatures: []

Keep the antiscald/high-temperature-limit information in the installation/product notes where appropriate.

3. ACCESSIBILITY LIVE REGION

The current:
<section aria-live="polite">

causes the entire results panel to become the live region.

Replace this with a small dedicated status element for dynamic status messages, e.g.:

- "Optimizing your bathroom..."
- "Your design is ready."

Do not make the entire recommendation/results panel aria-live.

After these changes run:
npm run lint
npm test
npm run build

Do not modify anything else.

- 18.** this is the output that Claude has given me
please verify it
[pasted the output of the Claude in the ChatGPT]

- 19.** One final correction before merge.

The catalog modeling of K-26341IN-NA is correct, but the current 2D layout renderer still rendersevery product as a floor rectangle using widthMM/depthMM.

That means the Anthem thermostatic valve can visually appear as a small floor fixture even though it is a concealed wall-mounted valve body.

Fix ONLY this visualization issue.

Requirements:

1. Keep K-26341IN-NA dimensions in the product catalog for reference/zone validation.
2. Do NOT change the optimizer.
3. Do NOT change the product data.

4. Do NOT remove the product from recommendations.

5. In components/bathroom-layout.tsx, detect:
product.subcategory === 'thermostatic-system'

6. Do not render it as a normal floor-footprint rectangle.

Instead render a small clearly labeled wall/control marker inside the shower zone, such as:
"THERMOSTATIC"

or

"THERMOSTATIC CONTROL"

Use a subtle outlined marker rather than a solid floor fixture footprint.

The marker should communicate:

- this is a wall/system component
- it belongs to the shower zone
- it is not a floor-mounted fixture

Keep the existing disclaimer:

"Zone-based feasibility visualization"

and

"not a certified architectural or CAD plan."

After this ONE change run:

npm run lint

npm test

npm run build

Do not change anything else.

20. please refer the output of the Claude and verify it and also tell me which files should I completely update with the given file of the Claude
[pasted the output of the Claude in the ChatGPT]

21. good everything is working but see the layout the dimensions are overlapping and not aligned see the top and right of the layout

One more thing that my complete website is looking white and black only
please make it attractive website with the appropriate color theme

22. I want the background to look something like this which is present in the screenshot
[uploaded the screenshot of the theme]

23. now give me the final prompt that I will give to the Claude from our previous two chats
I have not given Claude any prompt after my recent commit

24. We are doing the FINAL VISUAL REFINEMENT pass for the KOHLER AI Bathroom Designer & Planner.

IMPORTANT CONTEXT:

- The current repository already contains the finalized Track 1 functionality.
- The latest implementation has already been committed to Git.
- The smart toilet and thermostatic-system product support is already implemented.
- The optimizer and sustainability engine are already working.
- The UI has already received the first accessibility/button/animation polish.
- The current local project has already passed:
npm run lint
npm test
npm run build
- DO NOT redo or redesign any of those systems.

Your job in this task is ONLY to improve:

1. the 2D bathroom plan dimension annotations
2. the overall visual theme and background
3. the visual hierarchy and premium appearance

Do not change the application architecture or business logic.

=====

FIRST: INSPECT THE CURRENT REPOSITORY

=====

Before editing anything, inspect:

- app/page.tsx
- app/globals.css
- components/bathroom-layout.tsx
- components/ui/*
- data/products.ts

Also inspect the current git diff/status.

Do NOT make changes until you understand the existing implementation.

=====

GOAL 1 — FIX THE 2D BATHROOM PLAN DIMENSIONS

=====

Current visual problem:

The bathroom dimensions in the 2D plan are not visually aligned.

For example:

- the top dimension such as "2134 mm" sits too close to or overlaps the top wall

- the right dimension such as "2743 mm" sits too close to or overlaps the right wall

Fix the dimension annotation system without changing the underlying bathroom dimensions, room geometry, fixture placement, optimizer, or layout calculations.

REQUIREMENTS:

A. TOP DIMENSION

- Put the width dimension clearly ABOVE the top room wall.
- Center it horizontally relative to the room.
- Give it a consistent gap from the wall.
- It must never overlap the room border.

B. RIGHT DIMENSION

- Put the height dimension clearly OUTSIDE the right room wall.
- Vertically center it relative to the room.
- Rotate the text -90 degrees so it reads along the vertical dimension.
- Give it a consistent gap from the wall.
- It must never overlap the room border.

C. DIMENSION GUIDES

Where practical, add subtle engineering-style extension lines from the room edges toward the dimension labels.

Keep them thin and muted.

D. DRAWING MARGINS

Use explicit annotation margins rather than arbitrary one-off pixel offsets.

Think in terms of:

top annotation margin
right annotation margin
left/bottom safety margin

The room itself should remain centered and correctly scaled.

E. RESPONSIVE BEHAVIOR

The solution must work for:

- small bathrooms
- large bathrooms
- desktop
- tablet
- narrow screens

Do not allow labels to collide with:

- room walls
- fixture labels
- room dimensions
- canvas boundaries

F. TYPOGRAPHY

Dimension labels should be:

- smaller than major section headings
- visually subtle
- muted gray/stone
- easy to read
- visually similar to professional interior-design/architectural annotations

Do NOT make dimension labels bold or visually dominant.

IMPORTANT:

- Do not change actual room dimensions.
- Do not change layout mathematics.
- Do not change the optimizer.
- Do not change the zone model.
- Do not change the thermostatic-system representation.
- Keep the existing disclaimer:
"Zone-based feasibility visualization"
and
"not a certified architectural or CAD plan."

=====

GOAL 2 — PREMIUM MARBLE BACKGROUND

=====

The current website is too white/black and feels visually flat.

I want the main website background to resemble a premium luxury white marble bathroom surface.

Use the supplied reference image as STYLE INSPIRATION ONLY.

REFERENCE STYLE:

- warm white / ivory marble
- subtle gray natural veins
- restrained beige/champagne/gold veins
- elegant luxury bathroom appearance
- Calacatta-style feel
- bright and sophisticated
- natural irregular veining
- subtle stone texture
- low contrast
- premium showroom feeling

DO NOT:

- use the exact reference image
- use a watermarked stock image
- introduce a copyright/watermark
- use dark marble
- use strong black veins
- use busy/high-contrast marble

- make the background distracting

Create/use an ORIGINAL marble-like background.

PREFERRED IMPLEMENTATION:

- a local asset inside public/images/
- optimized for web
- no external image URL
- no external runtime dependency

If using a generated/local image is not practical, use a subtle CSS-based marble-like treatment instead.

=====

GOAL 3 — PAGE COLOR THEME

=====

Build a restrained premium bathroom-design palette around the marble.
Use this approximate design system:

BACKGROUND / MARBLE BASE
#F7F5F0

SURFACES / CARDS
#FFFDFC

PRIMARY TEXT
#292521

SECONDARY TEXT
#716B64

BORDER
#DED7CD

PRIMARY ACCENT / BRONZE
#96623F

PRIMARY ACCENT HOVER
#7D4F32

SUSTAINABILITY / SAGE
#668878

WARNING
#B47A2A

ERROR
#B54A47

IMPORTANT:

These are visual design directions, not claims about an official KOHLER brand palette.

Use CSS variables/design tokens in app/globals.css where practical instead of scattering hardcoded colors throughout JSX.

=====

GOAL 4 — VISUAL HIERARCHY

=====

The final experience should feel like a premium digital bathroom-design product.
Visual hierarchy should be approximately:

1. Generate My Bathroom
2. Recommended design
3. Product information
4. Water/sustainability impact
5. Supporting controls

The marble background is decorative.
The content must remain the focus.

=====

CARDS

=====

Replace the flat black/white feeling with subtle depth:

- warm white surfaces
- soft beige borders
- subtle shadows
- refined corner radius
- restrained spacing
- slight separation from the marble background

Avoid:

- huge shadows
- glassmorphism
- neon effects
- excessive gradients
- overly rounded UI
- flashy visuals

=====

PRIMARY CTA

=====

Preserve the existing:

"Generate My Bathroom"

Do NOT rewrite its functionality.
Make it visually strong using the bronze accent:

- bronze background
- white text
- recognizable icon
- hover darkening
- active state
- focus-visible ring
- disabled state
- loading state

Keep the existing loading copy such as:
"Optimizing your bathroom..."
Do not remove existing accessibility behavior.

=====

SECONDARY ACTIONS

=====

Keep secondary actions visually lighter than the primary CTA.

Maintain clear button hierarchy:

Primary
Secondary
Tertiary

Keep readable text labels.
Do not rely on icons alone.

=====

AI INPUT AREA

=====

Make the AI interaction feel premium and intentional.
The AI section may use a very subtle warm-bronze tint or accent border.

Keep:

- Sparkles icon
- existing AI interpretation flow
- existing image analysis flow
- existing labels
- accessibility

Do not redesign the AI logic.

=====

IMAGE UPLOAD

=====

Keep the existing image workflow exactly as it works.

Only improve presentation:

- warm-white surface
- subtle border
- clear upload control
- bronze accent for active/important action
- readable file state
- preview remains clear

=====

RESULTS

=====

The results should look like a premium design recommendation rather than a plain dashboard.

Improve:

- recommended bundle card
- price visibility
- budget utilization
- recommendation score
- product cards
- water savings
- 2D layout

The top recommendation should remain visually distinct.

Do NOT introduce a new ranking/scoring method.

=====

WATER SAVINGS

=====

Keep the current sustainability logic completely unchanged.

Style the sustainability section with muted sage/teal.

Example visual hierarchy:

WATER SAVED
81,030 L / year

Estimated under your stated usage assumptions

Use:

- sage/teal surface
- Droplets icon
- strong numeric emphasis
- smaller assumption/disclaimer text

Do not claim exact real-world savings.

=====

WORKFLOW INDICATOR

=====

Preserve:

- 01 Requirements
- 02 Optimization
- 03 Your Design

Use:

- neutral treatment for inactive stages
- bronze accent for active/completed stages
- subtle borders
- readable typography

Keep it compact and responsive.

=====

2D BATHROOM PLAN VISUAL STYLE

=====

Keep the diagram itself clean.

Recommended visual treatment:

ROOM:

warm white / very light stone

ROOM BORDER:

deep charcoal or muted stone

FIXTURES:

deep charcoal / warm dark neutral

IMPORTANT/ACTIVE ELEMENT:

bronze accent where useful

THERMOSTATIC MARKER:

retain the existing distinct thermostatic marker
and do not turn it back into a normal floor fixture

DIMENSIONS:

muted gray/stone
professional annotation style

Do not make the 2D plan overly colorful.

=====

BACKGROUND APPLICATION

=====

The marble treatment should primarily appear behind the page content.
Major cards should have mostly solid warm-white surfaces.

Do NOT put strong marble texture directly behind:

- dense form controls
- product cards
- recommendation text
- the 2D plan

The result should feel like:

Luxury marble background

+

warm-white cards

+

bronze primary actions

+

sage sustainability accents

+

charcoal typography

=====

ACCESSIBILITY

=====

Preserve the existing accessibility work.

Do not regress:

- focus-visible states
- aria-live status behavior
- image upload labeling
- keyboard accessibility
- icon aria-hidden behavior
- readable contrast
- reduced-motion support

The visual redesign must not reduce accessibility.

=====

RESPONSIVENESS

=====

Check:

- desktop
- tablet
- narrow mobile

Ensure:

- no horizontal overflow
- buttons remain usable
- cards remain readable
- marble background scales correctly
- 2D canvas remains contained
- dimension annotations remain outside the room
- dimension labels do not overlap the canvas edges

=====

PERFORMANCE

=====

Do not add a heavy animation or background library.

If adding a marble image:

- keep it local
- optimize its dimensions/file size
- avoid unnecessary large assets
- do not create multiple redundant background assets

=====

STRICT SCOPE

=====

DO NOT CHANGE:

- Gemini behavior
- AI prompts
- optimizer
- scoring logic
- sustainability mathematics
- product catalog data
- product verification
- API contracts
- route handlers
- test architecture
- database architecture
- layout-generation mathematics
- category logic

This is a VISUAL/UI REFINEMENT task.

Only modify the files/assets actually necessary.

=====

VALIDATION

=====

After editing, run:

npm run lint

npm test

npm run build

All three must pass.

If any fail:

- diagnose

- fix

- rerun

Do not finish with errors.

Also inspect:

git diff --stat

git status

Do not commit automatically.

=====

FINAL REPORT

=====

When finished, report:

1. Files changed
2. How the 2D dimension positioning was fixed
3. Marble background implementation
4. Final color palette
5. UI hierarchy improvements
6. Accessibility preserved
7. Responsive behavior checked
8. npm run lint result
9. npm test result
10. npm run build result
11. Any remaining limitations

FINAL RULE:

This must look like a polished premium bathroom-design application, not a generic black-and-white developer dashboard.

The marble should be elegant and subtle.

The bronze should provide the primary visual identity.

The sage/teal should communicate sustainability.

The UI must remain readable and practical.

25. This is the output given by the Claude
please verify it and also tell me which files need to be updated and changed
[pasted the output of the Claude in the ChatGPT]
26. please help me add more Kohler products like we only have 2-2 products for each now i want 10 -10
products for each
please help me do that with the help of Claude
27. We are doing a CONTROLLED CATALOG EXPANSION for the KOHLER AI Bathroom Designer & Planner.

IMPORTANT:

The current application is working and the current main branch is already stable.

DO NOT redesign the architecture.

DO NOT change the AI/deterministic boundary.

DO NOT change the optimizer methodology.

DO NOT change sustainability mathematics.

DO NOT modify API contracts.

DO NOT modify the UI unless absolutely required to display the expanded catalog.

DO NOT replace existing products.

DO NOT invent product data.

The goal is to expand the verified KOHLER product catalog from the current small MVP catalog to
EXACTLY:

10 TOILETS

10 FAUCETS

10 SHOWERS

10 VANITIES

= 40 total distinct products.

=====

PHASE 1 — INSPECT CURRENT CATALOG

=====

First inspect:

- data/products.ts
- data/products.backup.ts
- types/product.ts
- data/schemas.ts
- data/categories.ts
- lib/engine/optimizer.ts
- lib/engine/constraints.ts
- lib/engine/scoring.ts
- lib/engine/sustainability.ts

- __tests__/*

Determine the CURRENT exact number of products in each category.

Do not assume the count from previous messages.

Create an internal inventory:

TOILETS:

existing model numbers

FAUCETS:

existing model numbers

SHOWERS:

existing model numbers

VANITIES:

existing model numbers

Then calculate exactly how many NEW products are required to reach 10 per category.

DO NOT duplicate an existing model number.

=====

PHASE 2 — OFFICIAL KOHLER PRODUCT RESEARCH

=====

Research CURRENT KOHLER INDIA products.

Use official KOHLER India product pages:

<https://www.kohler.co.in/>

Only use current products that can be verified from KOHLER India.

The product page/product documentation should be the source of truth.

DO NOT use:

- Amazon
- Flipkart
- random retailer sites
- scraped product databases
- unofficial blogs
- fabricated product specifications

Other sources may be used only to locate a product, but critical product attributes must be verified against KOHLER India or official KOHLER documentation.

=====

PHASE 3 — PRODUCT SELECTION

=====

Select DISTINCT products, not merely different finish variants of the same model.

Each product should have a unique model/SKU.

Prioritize diversity of actual product types.

TOILETS:

Target 10 total.

Include a sensible mix where available:

- smart/intelligent toilet
- one-piece toilets
- wall-hung toilets
- two-piece toilets
- wall-hung with exposed tank

The catalog MUST retain the existing smart toilet:

K-29777IN-0

Do not remove or replace it.

Use accurate subcategories.

Examples of currently available KOHLER India toilet families/types include smart toilets, wall-hung toilets and one-piece toilets, but verify every selected SKU individually before adding it.

FAUCETS:

Target 10 total.

Prefer meaningful product diversity:

- single-control
- tall basin faucet
- wall-mount
- widespread
- different KOHLER collections

Do not simply add the same faucet with different finish codes.

Keep the existing:

K-38467IN-4ND-CP

K-38886IN-4ND-CP

SHOWERS:

Target 10 total.

Use meaningful diversity:

- showerheads
- handshowers
- rainheads/rainpanels
- complete shower kits where appropriate
- thermostatic systems/components

The catalog MUST retain:

K-72439IN-EC-CP

K-26286IN-G-AF

K-26341IN-NA

IMPORTANT:

K-26341IN-NA is a mechanical thermostatic valve component, not a complete shower fixture.

Keep its accurate subcategory:

thermostatic-system

Never represent a thermostatic valve as a floor-mounted fixture.

Where KOHLER states that a compatible control-panel trim is sold separately, preserve that disclosure.

Do not call a showerhead a "thermostatic shower" unless the official KOHLER product documentation actually identifies thermostatic functionality.

VANITIES:

Target 10 total.

Prefer diversity by:

- width
- style/collection
- wall-hung vs other applicable configurations
- bathroom size suitability

Keep existing:

K-31602IN-E64

K-31601IN-E64

Do not create fictitious vanity sizes.

=====

PHASE 4 — DATA QUALITY RULES

=====

EVERY new product must have:

- unique id
- modelNumber
- name
- category
- subcategory
- priceINR
- dimensions
- installation
- style
- productUrl
- dataConfidence

- verificationDate

Water consumption:

For toilets:

use official LPF/dual-flush information when available.

For faucets:

use official LPM information when available.

For showers:

use official LPM information when available.

For vanities:

waterConsumption should remain null because a vanity itself is not a water-flow appliance.

DO NOT invent values.

If an official product page does not expose a critical specification:

- search the official KOHLER technical/specification document
- if still unavailable, use the existing schema's nullable/estimated approach where permitted
- do not fabricate numbers.

=====

PRICE RULE

=====

Use the CURRENT KOHLER INDIA displayed price in INR.

Do not copy an old price from search snippets if the current product page displays another price.

Use the price actually published by KOHLER India at implementation time.

Record the verification date.

=====

DIMENSION RULE

=====

Dimensions are extremely important because the optimizer uses them.

For every new product:

- verify dimensions from the official product page or official specifications
- preserve the actual width/depth/height semantics
- convert inches/cm to millimetres where necessary
- do not guess which axis is width/depth/height

If the official documentation is ambiguous, do not invent the mapping.

Use dataConfidence = estimated only when appropriate and make the limitation explicit.

=====

INSTALLATION RULE

=====

Preserve installation information accurately.

Examples:

- floor-mount
- wall-hung
- deck-mount
- wall-mount
- ceiling-mount
- concealed
- etc.

Do not claim compatibility unless KOHLER explicitly documents it.

For systems/components, use installation notes to explain dependencies.

=====

STYLE MATCHING

=====

Assign existing StyleTheme values conservatively:

- minimalist-modern
- classic-luxury
- japanese-zen
- contemporary
- transitional
- industrial

Do not create new style values unless absolutely necessary.

Base style assignment on the actual design/collection characteristics.

=====

PRODUCT IMAGES

=====

Do NOT download or create fake product images.

If the existing product schema supports official image URLs and a reliable KOHLER India image URL is available, use it.

Otherwise preserve the existing placeholder/top-view SVG approach.

Do not break the UI just because a product image is unavailable.

The priority is verified product metadata and recommendation quality.

=====

PHASE 5 — CATALOG STRUCTURE

=====

Keep the existing TypeScript/Zod architecture.

DO NOT introduce:

- a database
- Supabase
- a product microservice
- a CMS
- a new backend

Continue using:

data/products.ts

and existing Zod validation.

The catalog should remain static and type-safe for this MVP.

=====

PHASE 6 — OPTIMIZER PERFORMANCE

=====

The current optimizer enumerates category combinations.

With:

10 toilets

× 10 faucets

× 10 showers

× 10 vanities

there are at most:

10,000 raw combinations.

This is completely acceptable for this MVP.

Do not replace the deterministic optimizer with AI.

Verify that:

- all 10 products in each category participate in candidate generation
- hard budget constraints still work
- physical zone constraints still work
- top-K ranking still works
- sustainability still works
- no product category gets silently dropped

Do not artificially force specific products into recommendations.

=====

PHASE 7 — PRODUCT VALIDATION TESTS

=====

Add or update tests.

The test suite must verify:

1. Exactly 10 toilets exist.
2. Exactly 10 faucets exist.
3. Exactly 10 showers exist.
4. Exactly 10 vanities exist.
5. All model numbers are unique.

6. All product IDs are unique.
7. All products pass the Zod/catalog validation.
8. Existing smart toilet remains present.
9. Existing thermostatic-system component remains present.
10. Optimizer can consider all expanded categories.
11. Budget hard constraint still works.
12. Physical fit constraint still works.
13. Existing no-feasible behavior still works.
14. Sustainability calculation still works.
15. At least one optimizer test proves products beyond the original 2-product catalog can actually be selected.

Do NOT simply write tests that assert numbers without validating the underlying product records.

=====

PHASE 8 — WATER DATA QUALITY

=====

The expanded catalog must remain honest for sustainability scoring.

Rules:

Known water consumption:

→ calculate normally.

Unknown water consumption:

→ preserve null.

Unknown water consumption must NOT receive perfect efficiency.

The existing neutral handling for missing water data must remain.

Do not modify the sustainability algorithm just to make new products score better.

=====

PHASE 9 — PRODUCT RECOMMENDATION QUALITY

=====

After expansion, manually test several scenarios:

A.

7 × 9 ft

₹150,000

Minimalist Modern

4 people

B.

8 × 10 ft

₹250,000

Contemporary

4 people

C.

10 × 12 ft

₹600,000
Classic Luxury
5 people

D.
Small bathroom
low budget

E.
Large bathroom
high budget

Verify that different products can appear across different scenarios.
The objective is not merely to have 40 records.
The objective is to make the additional catalog actually useful to the optimizer.

=====
PHASE 10 — DO NOT BREAK EXISTING UI
=====

The current UI already has:

- marble background
- bronze accent
- sage sustainability styling
- improved primary CTA
- accessibility improvements
- 2D layout
- thermostatic-system visualization

Do not redesign these features during catalog expansion.
Only make UI changes if an expanded catalog causes a real rendering or data issue.

=====
PHASE 11 — VALIDATION
=====

Run:
npm run lint
npm test
npm run build

All three MUST pass.
Do not finish with:

- lint errors
- failing tests
- TypeScript errors
- build errors

Also run:
git diff --stat

git status

Do not commit automatically.

=====
PHASE 12 — FINAL REPORT
=====

Report:

1. Existing product count before changes
2. Final product count by category
3. Total final catalog count
4. Every newly added model number
5. Official KOHLER India URL for every newly added product
6. Which products are smart toilets
7. Which products are thermostatic systems/components
8. Which products have verified water-consumption data
9. Which products have estimated/null fields and why
10. Test count and result
11. Lint result
12. Build result
13. Files changed
14. Any remaining limitations

CRITICAL:

Do not claim "10 verified products per category" unless every one has actually been checked against KOHLER India/official KOHLER documentation.

If fewer than 10 suitable products can be verified for a category, STOP at the verified count and report exactly what is missing instead of fabricating products.

DO NOT commit automatically.

DO NOT push automatically.

28. till the Claude generate its output help me with the deployment of this

29. till the Claude is working on the assigned task lets build the production ready readme file of this

30. check the output of the claude
[pasted the output of the Claude in the ChatGPT]

31. Your current work is technically sound, especially the decision not to fabricate dimensions.
However, the assigned catalog-expansion task is NOT complete yet.

Do NOT stop at 12 products.

The target remains:

10 toilets
10 faucets
10 showers
10 vanities
40 total

Continue the research in the same disciplined way.

IMPORTANT:

Do not invent dimensions or other specifications.

However, your previous search appears to have treated the individual KOHLER product page as the only possible source of dimensions. That is too restrictive.

For each candidate, search ALL official KOHLER sources, including:

- KOHLER India product page
- official KOHLER India technical specification PDF
- official KOHLER India price book
- official KOHLER India retail catalogue
- official KOHLER technical documentation hosted on techcomm.kohler.com
- official KOHLER regional documentation when it is clearly the same model/SKU

The product page does not have to contain every dimension itself if an official KOHLER technical document provides the missing dimension.

Do NOT use:

- Amazon
- Flipkart
- retailer product pages
- random blogs
- scraped catalogues
- unofficial databases

For every added product, record the exact official source used for:

- price
- dimensions
- water consumption where applicable
- installation
- product identity

Prioritize completing the categories systematically.

CURRENT COUNTS:

Toilets: 5
Faucets: 2
Showers: 3

Vanities: 2

REMAINING:

Toilets: 5

Faucets: 8

Showers: 7

Vanities: 8

You have already identified promising faucet candidates such as:

- K-25757IN-4ND-CP
- K-33059IN-4ND-CP

Continue investigating these rather than leaving them unused if the official technical documents provide the missing envelope.

Also investigate the additional faucet SKUs visible in official KOHLER India catalog material, provided each model is still appropriate/current and its current India pricing/product identity can be verified.

For toilets, continue investigating the additional KOHLER India models you already identified, such as K-33123IN-0, and other current one-piece, two-piece, wall-hung and smart toilets.

For showers, prioritize:

- showerheads
- handshowers
- rainheads/rainpanels
- body sprays where they fit the current shower-category model
- thermostatic systems/components

Do not label a normal showerhead as thermostatic.

For vanities, prioritize genuinely different:

- dimensions
- collections
- configurations
- sizes

Do not count different finish variants of one model as separate products.

TARGET:

Reach exactly 10 unique model/SKU records per category if sufficient officially verifiable products exist.

If a category genuinely has fewer than 10 suitable/current products after exhaustive official-source research, report the verified maximum and explain exactly which missing fields prevent more products.

Do not fabricate data to reach 40.

DATA QUALITY:

Every added product must have:

- unique id
- unique modelNumber
- name
- category
- subcategory
- current INR price
- width/length/height or whatever the existing schema requires
- installation
- style
- productUrl
- verificationDate
- dataConfidence

For water data:

- use official LPF/LPM where documented
- otherwise keep null
- never treat missing data as perfect efficiency

TESTS:

Continue extending the tests.

Maintain:

- uniqueness validation
- catalog count validation
- zone-fit validation
- budget validation
- category coverage
- optimizer selection
- sustainability behavior

Run:

```
npm run lint
npm test
npm run build
```

The previous sandbox build limitation does not remove the requirement. If your environment cannot run the full build, state that clearly, but ensure tsc/lint/tests are clean.

Do not commit or push.

FINAL REPORT:

Give me the final inventory grouped as:

TOILETS — 10

model | name | price | dimensions | water data | confidence | official URL

FAUCETS — 10

same fields

SHOWERS — 10

same fields

VANITIES — 10

same fields

Then report:

- total products
- source verification method
- tests
- lint
- build
- remaining limitations

Do not stop merely because the first few searches lack dimensions. Search the official technical documentation and price books before declaring a product blocked.

- 32.** Verify the output given by Claude and suggest some changes if needed
[pasted the output of the Claude in the ChatGPT]